PROJECT REVIEW PREP NOTES

Pharmacy Inventory & Prescription Management System

Database design, concepts used, and answers to likely review questions

What changed since the last version

This version drops the frontend as the main focus and puts the database layer at the centre, since that's what's being evaluated this review. A third database, Cassandra, has been added to demonstrate polyglot persistence: three databases, three different jobs, three different concepts to defend.

Sarvaswa | 24BCE0333 | B.Tech CSE, VIT Vellore

OVERVIEW

The Workflow, End to End

One dispensing action touches three databases, each for a different reason

The one-line pitch

When a pharmacist dispenses a medicine, PostgreSQL handles the transaction and picks the right batch, MongoDB checks whether the combination is safe, and Cassandra logs the event for analytics and audit. None of the three could do all three jobs well on its own, and that's the point of the design.

1. PostgreSQL

Medicines, batches, prescriptions. Handles the transaction: lock the row, pick the batch closest to expiry, update stock.

2. MongoDB

Drug interactions and guidelines. Checked right before the dispense is confirmed, warns if the combination is risky.

3. Cassandra

Dispensing event log. Every dispense, stock adjustment, and batch receipt gets written here as an append-only record.

WHY THIS COUNTS AS "POLYGLOT PERSISTENCE"

Polyglot persistence means picking a different database for each part of a system based on how that data actually behaves, instead of forcing everything into one engine. Here, inventory data needs strong consistency and locking because it involves real stock and real money. Interaction data changes shape often and is read far more than it's written, so a flexible document store fits better. Event logs are append-only and enormous in volume over time, which is exactly what a wide-column store like Cassandra is built for. Three workloads, three engines, each chosen on purpose.

The flow, step by step

- Pharmacist searches for a medicine against a prescription
- PostgreSQL locks the relevant batch row and picks the one nearest to expiry (FEFO)
- Backend queries MongoDB for interactions between all medicines on that prescription
- If a conflict is found, a warning is shown before the pharmacist can proceed
- On confirmation, PostgreSQL commits the transaction and updates stock
- Cassandra receives a write for the dispensing event, independent of the transaction above

DATABASE ONE

PostgreSQL: The Transactional Core

Where correctness and consistency matter more than anything else

Why PostgreSQL here:

Inventory counts and prescriptions are relational by nature and directly tied to stock and billing. If two people can dispense the last unit of a batch at the same time, that's a real-world bug, not a theoretical one. Postgres gives ACID guarantees that make this impossible.

ACID transactions

The whole dispense operation, from checking stock to updating quantity, runs inside one transaction. If any step fails, everything rolls back. Nothing is left half-updated.

Row-level locking

`SELECT ... FOR UPDATE` locks the batch row being read. A second pharmacist trying to dispense from the same batch has to wait until the first transaction commits or rolls back.

Indexing for FEFO

A B-tree index on `expiry_date` within each medicine's batches means the "pick the earliest expiring batch" query stays fast even as batch history grows.

Referential integrity

Foreign keys tie batches to medicines and prescriptions to medicines. You can't insert a batch for a medicine that doesn't exist, and the database enforces that, not the application code.

The query he'll probably want to see

```
SELECT * FROM batches
WHERE medicine_id = $1 AND quantity > 0
ORDER BY expiry_date ASC
LIMIT 1
FOR UPDATE;
```

This is the FEFO query. It's also the query that needs the lock, because two transactions could otherwise both read the same row before either commits.

DATABASE TWO

MongoDB: The Safety Knowledge Layer

Where the data doesn't fit neatly into rows and columns

Why MongoDB here:

Drug interaction data comes from external medical sources and doesn't have a fixed shape. Some interactions have dosage thresholds, some don't. Some guidelines have five fields, some have twelve. Forcing this into rigid tables means constant schema migrations. A document model absorbs that variability naturally.

Schema flexibility

Each document in `drug_interactions` can carry different fields depending on what's known about that pair of drugs, without altering a table definition.

Document model vs relational

One interaction record maps to one real-world fact. There's no need to join across multiple tables just to answer "do these two drugs conflict."

Indexing on drug pairs

A compound index on `drug_a` and `drug_b` keeps the interaction check fast, since this query runs on every single dispense.

Read-heavy workload

This collection is updated occasionally as new interactions are published, but read on nearly every transaction. Mongo handles that read pattern well.

Sample document

```
{
  drug_a: "Aspirin",
  drug_b: "Warfarin",
  severity: "high",
  effect: "Increased bleeding risk",
  recommendation: "Avoid combination"
}
```

DATABASE THREE

Cassandra: The Event Log

Added to show the write-heavy, distributed side of the design

Why Cassandra here:

Every dispense, stock adjustment, and batch receipt generates an event. Across a chain of pharmacies, that's a very high volume of small, append-only writes that are almost never updated after they're written. That's a bad fit for Postgres, which optimises for consistent reads and updates, and a good fit for Cassandra, which optimises for fast, distributed writes.

Wide-column model

Each row is identified by a partition key and a clustering key, rather than a single primary key. This is what makes Cassandra fast at scale for time-ordered data.

Partition & clustering keys

Partition by `medicine_id`, cluster by `event_timestamp`. This means "show me every dispensing event for this medicine, most recent first" is a fast, single-partition read.

Tunable consistency

Cassandra lets you choose consistency level per query. For an event log, writing with a lower consistency level in exchange for speed is an acceptable trade, since this data isn't the source of truth for stock.

CAP theorem, in practice

Postgres here leans towards consistency. Cassandra leans towards availability and partition tolerance. Using both is a deliberate way to show that trade-off isn't just theoretical.

Making it more than a write sink

If there's time, the event log can also answer a query like "most dispensed medicines this week" by scanning recent partitions. That turns Cassandra from a place where data goes to die into something the system actually reads from, which is a better story for a review.

PUTTING IT TOGETHER

Comparison, and a Word on the Frontend

The database choices side by side, plus what the frontend will be, briefly

Aspect	PostgreSQL	MongoDB	Cassandra
Data shape	Rigid, relational	Flexible, document	Wide-column, time-ordered
Optimised for	Consistency, transactions	Read-heavy, flexible reads	Write throughput, scale
CAP leaning	Consistency first	Consistency, tunable	Availability first
Used for	Medicines, batches, prescriptions	Interactions, guidelines	Dispensing event log

FRONTEND, NOT THE FOCUS BUT NOT IGNORED

Since this review is about the database layer, the frontend is being kept intentionally thin. A few points worth having ready:

- A single dispensing screen: search a medicine, see batch and expiry, see any interaction warning, confirm dispense
- Built later in React once the database logic and API layer are stable, so effort isn't split across both right now
- Could even start as a CLI or a simple set of API calls tested through Postman before any UI exists, since the logic is what's being graded
- Purpose of the frontend is to prove the workflow end to end, not to be a polished product this early

BE READY FOR THIS

Likely Review Questions: Database Design

Short, direct answers you can say out loud without reading off a slide

Why not just use one database for everything?	Because the three kinds of data behave differently. Inventory needs transactions and locks, interaction data needs a flexible schema, and event logs need to absorb a huge write volume. One engine would be a compromise on at least two of those.
Why not store interactions as JSON inside Postgres instead of using Mongo?	Postgres can store JSON, but that doesn't give the read-optimised indexing or the horizontal scalability Mongo gives for a collection that's queried on nearly every dispense. It also keeps the safety data's lifecycle independent of the transactional database.
How do you handle two pharmacists trying to dispense the last unit of a batch at the same time?	The batch row is locked with `SELECT ... FOR UPDATE` inside a transaction. The second pharmacist's query waits until the first transaction commits or rolls back, so the stock count never goes negative or gets double-counted.
What happens if MongoDB is down when a dispense is attempted?	The safer default is to block the dispense and show that the interaction check couldn't be completed, rather than silently skipping the safety check. This is a design decision worth stating clearly if asked.
How is "closest to expiry" decided?	A query on the batches table for that medicine, filtered to batches with stock remaining, sorted by expiry_date ascending, taking the first result. That's the FEFO rule, first expired, first out.
Is the schema normalised?	Yes for the Postgres side. Medicines, batches, and prescriptions are separate tables linked by foreign keys, so a medicine's details aren't repeated across every batch row.

BE READY FOR THIS

Likely Review Questions: Cassandra & Scale

This is the newer part of the design, so expect more digging here

Why did you add a third database, isn't two enough?	Two databases already cover the core requirement. Cassandra was added to demonstrate a third real pattern, append-only, high-volume event data, since that's a workload the other two don't handle as well and it strengthens the "right tool for the job" argument.
What exactly does Cassandra store here?	A log of dispensing events, stock adjustments, and batch receipts. It's not the source of truth for current stock, Postgres is. Cassandra is a record of what happened, over time.
What's a partition key and why does it matter?	It decides which node in a Cassandra cluster stores a given row. Partitioning by medicine_id means all events for one medicine live together, so querying that medicine's history doesn't scan the whole cluster.
What's the CAP theorem angle here?	CAP says a distributed system can't fully guarantee consistency, availability, and partition tolerance all at once. Postgres in this design favours consistency for money-adjacent data. Cassandra favours availability and write speed for the event log, since a slightly stale read on historical events isn't a real problem.
Isn't this overengineered for a student project?	It's more than the minimum, on purpose, because the goal of this review is to demonstrate database concepts, not just deliver the smallest working system. Each database earns its place with a specific reason, which is the point being made.